

Slide 1 - Project Objective

Our project is a distributed file system for text-oriented files, inspired by the Google File System architecture.

The goal is to make file storage look simple from the client's point of view, while internally the system distributes the data across multiple storage servers.

The system is based on three distributed-systems concepts. First, sharding: files are divided into fixed-size chunks. Second, replication: each chunk is stored on three different storage servers. Third, separation of concerns: metadata is managed separately from the actual file content.

From the user perspective, the client supports normal file operations: create, read, delete, size, and list. The client hides where chunks are stored and how replicas are selected.

Slide 2 - System Architecture

The architecture has three layers: the client, the naming server, and the storage servers.

The naming server is the single metadata authority. It stores the namespace, the file-to-chunk mapping, the order of chunks, and the replica locations for each chunk. This metadata is persisted in SQLite, so a naming-server process restart can recover the file metadata.

The storage servers store the real chunk bytes as files on disk. They do not store global metadata and they do not need to know the complete file structure. Each storage server holds only the chunks assigned to it.

The client coordinates the operation. It talks to the naming server for metadata and placement decisions, but it transfers chunk data directly to and from the storage servers using gRPC. This avoids routing bulk file content through the naming server.

In the final Docker topology, we run one naming server, four storage servers, and one client helper container.

Slide 3 - Data Handling

The system is designed for text-oriented files. Internally, the client treats file content as bytes and splits it into fixed chunks of 1,024 bytes. The last chunk can be smaller if the file size is not an exact multiple of one kilobyte.

Each chunk receives a unique UUID-style identifier. This identifier is stored in the metadata and is also used as the chunk filename on storage-server disks.

The final configuration uses a replication factor of three. That means every chunk is placed on three distinct storage servers. Placement is selected from the live storage servers using round-robin balancing.

Only metadata is stored in SQLite. The actual chunk content is stored as normal files under each storage server's data directory. This directly satisfies the requirement that the database is used only for metadata, not for file content.

Slide 4 - Create Operation

The create operation follows a four-stage pipeline.

First, the client splits the input file into one-kilobyte chunks and calculates the total size and number of chunks.

Second, the client calls `CreateFile` on the naming server. The naming server checks that enough storage servers are alive, creates a chunk identifier for each chunk, and returns a placement plan with three replica locations per chunk.

At this point, the file metadata is pending. The file is not yet readable, because the chunks may not have been fully written.

Third, the client sends each chunk directly to the selected storage servers using `StoreChunk`. Each storage server writes the chunk through a temporary file and then atomically replaces the final chunk file.

Finally, after all replicas have accepted the data, the client calls `CommitFile`. The naming server marks the file as committed, and only then does the file become visible to read operations.

This means an interrupted upload does not expose a partial file as valid.

Slide 5 - Read, Delete, and Size Operations

For a read operation, the client first requests the ordered chunk layout and replica addresses from the naming server. Then it downloads each chunk directly from a storage server and concatenates the chunks in index order.

If one replica is unavailable, the client tries another replica for the same chunk. The naming server also prioritizes live replicas when returning locations, which reduces failed attempts during reads.

For delete, the naming server asks every known replica location to remove its chunk, and then it removes the file metadata from SQLite. Chunk deletion is idempotent, so repeating a delete request for the same chunk is safe.

If a storage server is down during deletion, its chunk copy can remain on disk as an orphan. It is no longer reachable through the file system because the metadata has been deleted, but garbage collection is not implemented yet.

For size, the system answers only from metadata. It returns the total byte count and number of chunks without transferring any chunk content.

The system also includes list as a convenience operation for committed files.

Slide 6 - Fault Tolerance and Self-Healing

The fault-tolerance model is based on replication and heartbeat-based liveness detection.

Storage servers send heartbeats every five seconds. If the naming server does not receive a heartbeat for fifteen seconds, that storage server is considered unavailable.

With replication factor three, a chunk remains readable as long as at least one of its replicas is still available. If one storage server fails, reads continue through the remaining replicas. Since three storage servers are still alive, new writes can still create three distinct replicas.

If two storage servers fail, reads can still continue in this four-server topology, because every chunk has at least one surviving replica. However, new writes are rejected because only two storage servers are alive and the system cannot satisfy replication factor three.

If all three replicas of a chunk are unavailable, the chunk cannot be read. This is recoverable if a server comes back with its disk intact. If all three replica disks are permanently lost, the data is unrecoverable.

The naming server is also a single point of availability. If it is down, clients cannot locate chunks or perform operations. A process restart is recoverable because metadata is

stored in SQLite, but losing the metadata database would lose the file-to-chunk mapping.

The final system also includes automatic self-healing. Every five seconds, the naming server scans committed chunks and looks for chunks with fewer than three live replicas. If enough live capacity exists, it chooses a surviving source replica and a target storage server that does not already hold the chunk.

The target storage server copies the chunk from the source through `ReplicateChunk`. Metadata is updated only after the copy succeeds. This restores the target replication factor without requiring client intervention.

Slide 7 - Deployment and Verification

The system is Dockerized. Docker Compose starts the naming server, four storage servers, and a client helper container on the same internal network.

Only the naming-server port is exposed to the host. Storage servers use their internal Docker addresses for chunk I/O and replication.

The client is executed with `python -m gfs.client`. The commands map directly to the implemented operations: `create` uploads and shards a file, `read` reconstructs a file, `delete` removes metadata and replicas, `size` returns metadata-only size information, and `ls` lists committed files.

Verification is covered by six integration scenarios. They test multi-chunk create and read correctness, metadata-only size queries, reading after one storage-server failure, self-healing back to the replication factor, deletion, and rejection of writes when there are not enough live storage servers.

The broader fault-injection scenarios described in the testing plan are documented as future work, not as completed tests.

Slide 8 - Design Decisions and Conclusion

The main design choices are intentionally simple and consistent with the assignment.

The naming server is centralized, which keeps metadata management simple and avoids distributed coordination between metadata nodes. SQLite provides durable metadata persistence. Chunk content bypasses the naming server and moves directly between the client and the storage servers.

Writes use synchronous replication before commit. This increases write latency, but it gives a clear durability boundary: a file is only committed after all required replicas are stored.

The system also keeps the required fixed chunk size of one kilobyte, which makes placement and size calculation straightforward.

The main limitations are also clear. The naming server is a single point of availability. Synchronous replication makes writes slower than a single-copy write. The system does not yet implement garbage collection for orphaned chunks, chunk checksums, or strict UTF-8 validation.

In conclusion, the project implements the required distributed file system behavior. Files are split into fixed-size chunks, chunks are replicated across distinct storage servers, metadata is separated from content, and the client hides the physical distribution.

The self-healing mechanism extends the fault-tolerance model by restoring lost replicas after storage failures. The result is a working distributed file system that demonstrates both immediate failover through replication and recovery through automatic re-replication.